Lab 6: Arrays

Computer Programming (010153002) | Semester 1/2026

Duration: 2 hours (in-lab) | **Topic:** 1D and 2D arrays, traversal, accumulation, passing arrays to functions

Objectives

- Declare, initialize, and traverse 1D and 2D arrays.
- Pass arrays to functions and understand that the size is *not* carried with them.
- Implement basic array algorithms: search, min/max, reverse, frequency count.
- Use indices safely (bounds are 0 to `size` −1).

Quick Reference

Declaration and initialization.

```
int  a[5] = {1, 2, 3, 4, 5};
int  b[10] = {0};                /* all zeros */
int  m[3][4] = {{1,2,3,4},{5,6,7,8},{9,10,11,12}};
```

Bounds. For `a[N]` valid indices are `0..N-1`. Out-of-bounds access is *undefined behavior* – it may seem to work and still be wrong.

Passing to functions.

```
int sum(int a[], int n);    /* size must be passed separately */
```

Inside the function, `sizeof(a)` is *not* the array size – it is the size of a pointer.

Common idiom: max scan.

```
int max = a[0];
for (int i = 1; i < n; i++) if (a[i] > max) max = a[i];
```

Quick Experiments (Try It)

Try It 1: Partial initialization zeroes the rest

Run and observe every element:

```
#include <stdio.h>
int main(void) {
    int a[6] = {10, 20, 30};   /* only 3 initializers for 6 elements */
    for (int i = 0; i < 6; i++)
        printf("a[%d] = %d\n", i, a[i]);
    return 0;
}
```

1. What are the values of `a[3]`, `a[4]`, `a[5]`?
2. Change the initializer to `= {0}`. Does the whole array become zero?
3. Remove the initializer entirely (`int a[6];`). Are the values still zero? (Run it several times. Does the answer change?) What lesson does this teach?

Try It 2: Modifying an array inside a function affects the original

Run this and compare the output to Lab 5's pass-by-value behaviour:

```c
#include <stdio.h>
void fill(int a[], int n) {
    for (int i = 0; i < n; i++) a[i] = i * 10;
}
int main(void) {
    int v[5] = {0};
    printf("before fill: ");
    for (int i = 0; i < 5; i++) printf("%d ", v[i]);
    printf("\n");

    fill(v, 5);
    printf("after  fill: ");
    for (int i = 0; i < 5; i++) printf("%d ", v[i]);
    printf("\n");
    return 0;
}
```

1. Did `fill` change `v` in `main`?
2. In Lab 5, changing a plain `int` parameter inside a function had no effect on the caller. Why is an array different?
3. (Look ahead) Add `printf("%p\n", (void*)a);` inside `fill` and `printf("%p\n", (void*)v);` in `main`. Are the addresses the same?

Try It 3: Out-of-bounds access – silent and dangerous

Warning: this invokes undefined behaviour; save your work first.

```c
#include <stdio.h>
int main(void) {
    int a[3] = {1, 2, 3};
    for (int i = 0; i <= 5; i++)   /* intentional over-run */
        printf("a[%d] = %d\n", i, a[i]);
    return 0;
}
```

1. Does it compile? Does it crash immediately? What does it print for indices 3, 4, 5?
2. Run the program a second time. Are those out-of-bounds values the same?
3. Why is this so dangerous even when it seems to "work"?

In-Lab Exercises (target: 2 hours)

In-Lab Exercise 1: Read, Reverse, Print *(25 min)*

Read an integer N ($1 \le N \le 100$) and then N integers into an array. Print the array in reverse order on a single line.

In-Lab Exercise 2: Min, Max, Average *(25 min)*

Read an array of up to 50 doubles and compute the minimum, maximum, and average. Write three separate functions; do not duplicate the loop.

In-Lab Exercise 3: Linear Search *(30 min)*

Write `int find(int a[], int n, int key)` that returns the index of the first occurrence of `key` in the array, or `-1` if not present. Test from `main` with a key that is present and one that is not.

In-Lab Exercise 4: Matrix Transpose *(40 min)*

Read a 3×4 matrix of integers and print its 4×3 transpose. Use two nested `for` loops. Format each cell with field width 4 so columns align.

AI Collaboration

Try these prompts with an AI tool:

- "In C, what happens when I access `a[n]` on an array declared as `int a[n]`? What is undefined behavior?"
- "Why does `sizeof(a)` inside a function not give the array size when `a` is a parameter?"
- "Walk me through selection sort on `{5, 3, 8, 1}`, showing the array state after each pass."

Critical check – verify, don't just accept: Ask the AI to write a function that finds the maximum element in an array. Then call it with: an array of size 0 (n = 0), an array of size 1, and a normal array. Run the code and check whether it crashes, returns garbage, or handles these edge cases correctly. Note whether the AI warned you about them in advance.

Know the limit: AI tools routinely generate C array code without bounds checking and will not warn you that out-of-bounds access is undefined behavior – meaning it may silently produce wrong results rather than crashing, which is the most dangerous kind of bug.

Take-Home (Paper Work). Solve the following on paper without using a computer or compiler. Hand-write your answers; submit at the start of the next lab. Goal: prepare you to dry-code during the written exam.

Trace & Predict 1: Indexing

What does this print? (Watch the bounds!)

```c
#include <stdio.h>
int main(void) {
    int a[] = {10, 20, 30, 40, 50};
    int s = 0;
    for (int i = 1; i < 4; i++) s += a[i];
    printf("%d %d\n", s, a[0] + a[4]);
    return 0;
}
```

Trace & Predict 2: 2D traversal

Predict the printed output:

```c
#include <stdio.h>
int main(void) {
    int m[2][3] = {{1,2,3},{4,5,6}};
```

```
    for (int j = 0; j < 3; j++) {
        for (int i = 0; i < 2; i++) printf("%d ", m[i][j]);
        printf("\n");
    }
    return 0;
}
```

Find the Bug 1: Off-by-one + uninitialized accumulator

List all bugs and write the corrected program:

```
#include <stdio.h>
int main(void) {
    int a[5] = {2, 4, 6, 8, 10};
    int sum;
    for (int i = 0; i <= 5; i++) sum += a[i];
    printf("sum = %d\n", sum);
    return 0;
}
```

Write Code on Paper 1: Count occurrences

Write a complete program that reads N integers into an array ($N \leq 100$), then reads a target value and prints how many times the target occurs in the array.

Write Code on Paper 2: Reverse in place

Write a function `void reverse(int a[], int n)` that reverses the array *in place* (no auxiliary array). Then write a `main` that demonstrates it.

Draw on Paper 1: 2-D array memory layout

The declaration `int m[3][4]` stores elements in *row-major* order.

Part A – Grid view. Draw three rows of four boxes each. Label every box with its index notation (`m[0][0]`, `m[0][1]`, ..., `m[2][3]`).

Part B – Flat memory view. Below the grid, draw a single row of 12 consecutive boxes showing how the same elements are laid out in memory, and indicate the *byte offset* of each box relative to `m[0][0]` (assume `sizeof(int) = 4`).